

CO3-AT1-DAA

Name : N.MADHSUDHAN

Reg no : 1924245303

Question 22: Evaluate Quick Sort performance by implementing tail recursion optimization. Compare execution time with standard recursive implementation. Analyze stack usage and memory efficiency. Interpret the results and justify improvements achieved through optimization.

```
main.py Run Output Clear
1 import random
2 import time
3 import sys
4
5 sys.setrecursionlimit(50000)
6
7 def partition(a, low, high):
8     pivot = a[high]
9     i = low - 1
10    for j in range(low, high):
11        if a[j] <= pivot:
12            i += 1
13            a[i], a[j] = a[j], a[i]
14    a[i + 1], a[high] = a[high], a[i + 1]
15    return i + 1
16
17 max_depth_standard = 0
18 def quick_sort_standard(a, low, high, depth=0):
19     global max_depth_standard
20     if depth > max_depth_standard:
21         max_depth_standard = depth
22     if low < high:
23         p = partition(a, low, high)
24         quick_sort_standard(a, low, p - 1, depth + 1)
25         quick_sort_standard(a, p + 1, high, depth + 1)
26
27 max_depth_optimized = 0
28 def quick_sort_tail_optimized(a, low, high, depth=0):
29     global max_depth_optimized
30     while low < high:
31         if depth > max_depth_optimized:
32             max_depth_optimized = depth
33         p = partition(a, low, high)
```

```
--- Average Case (random input) ---
n=1000   Standard: time=0.00885s depth=20   Optimized: time=0.000918s depth=6
n=5000   Standard: time=0.007220s depth=24   Optimized: time=0.005415s depth=7
n=10000  Standard: time=0.012626s depth=29   Optimized: time=0.011233s depth=8
n=20000  Standard: time=0.026238s depth=30   Optimized: time=0.030478s depth=8
n=50000  Standard: time=0.068898s depth=40   Optimized: time=0.072470s depth=9

--- Worst Case (already sorted input, n=5000) ---
Standard Recursive : time=0.914437s max_recursion_depth=4999
Tail-Optimized      : time=0.894598s max_recursion_depth=0

=== Code Execution Successful ===
```

```
main.py Run Output Clear
27 max_depth_optimized = 0
28 def quick_sort_tail_optimized(a, low, high, depth=0):
29     global max_depth_optimized
30     while low < high:
31         if depth > max_depth_optimized:
32             max_depth_optimized = depth
33         p = partition(a, low, high)
34         if p < low < high - p:
35             quick_sort_tail_optimized(a, low, p - 1, depth + 1)
36             low = p + 1
37         else:
38             quick_sort_tail_optimized(a, p + 1, high, depth + 1)
39             high = p - 1
40
41 sizes = [1000, 5000, 10000, 20000, 50000]
42 for n in sizes:
43     original = [random.randint(1, 1000000) for _ in range(n)]
44
45     a1 = original.copy()
46     max_depth_standard = 0
47     start = time.perf_counter()
48     quick_sort_standard(a1, 0, len(a1) - 1)
49     t_std = round(time.perf_counter() - start, 6)
50
51     a2 = original.copy()
52     max_depth_optimized = 0
53     start = time.perf_counter()
54     quick_sort_tail_optimized(a2, 0, len(a2) - 1)
55     t_opt = round(time.perf_counter() - start, 6)
56
57     print("n = ", n, " Standard depth:", max_depth_standard, " time:", t_std)
58     print("n = ", n, " Optimized depth:", max_depth_optimized, " time:", t_opt)
```

```
--- Average Case (random input) ---
n=1000   Standard: time=0.00885s depth=20   Optimized: time=0.000918s depth=6
n=5000   Standard: time=0.007220s depth=24   Optimized: time=0.005415s depth=7
n=10000  Standard: time=0.012626s depth=29   Optimized: time=0.011233s depth=8
n=20000  Standard: time=0.026238s depth=30   Optimized: time=0.030478s depth=8
n=50000  Standard: time=0.068898s depth=40   Optimized: time=0.072470s depth=9

--- Worst Case (already sorted input, n=5000) ---
Standard Recursive : time=0.914437s max_recursion_depth=4999
Tail-Optimized      : time=0.894598s max_recursion_depth=0

=== Code Execution Successful ===
```